Two Dimensional Array

Two Dimensional Array (2D)

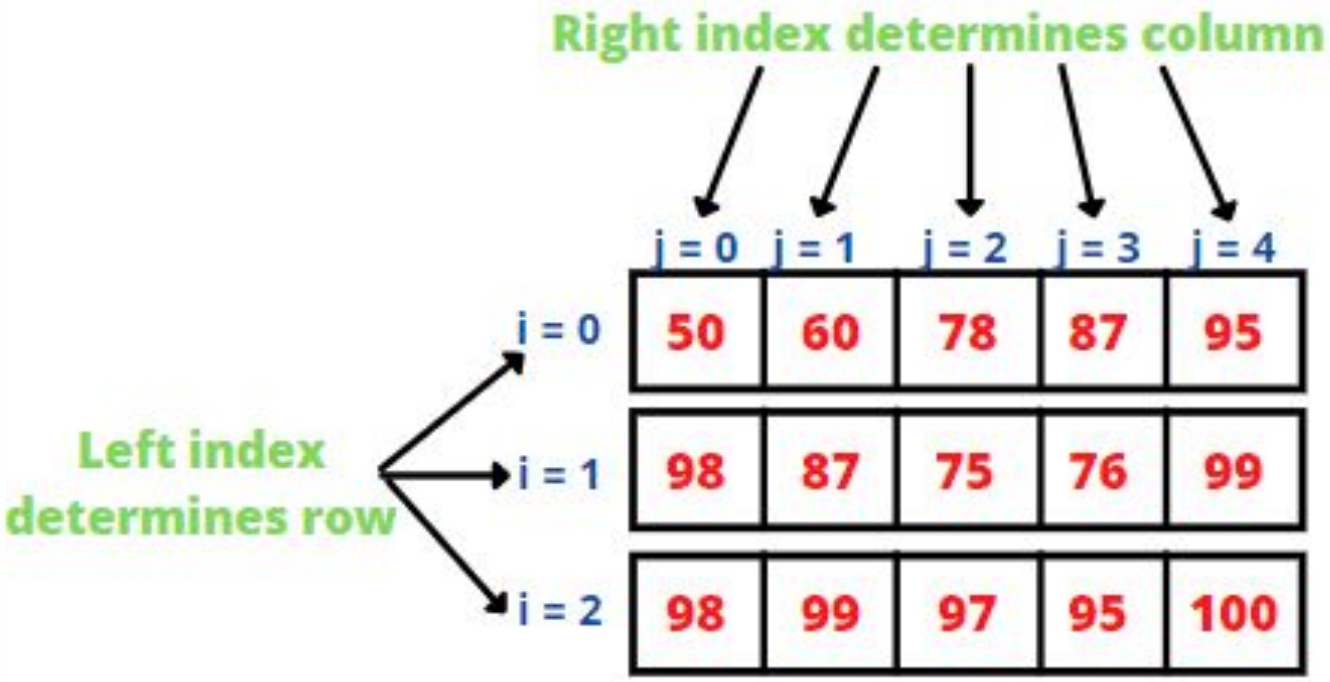

Fig: Arrangement of elements in two dimensional array

Contd...

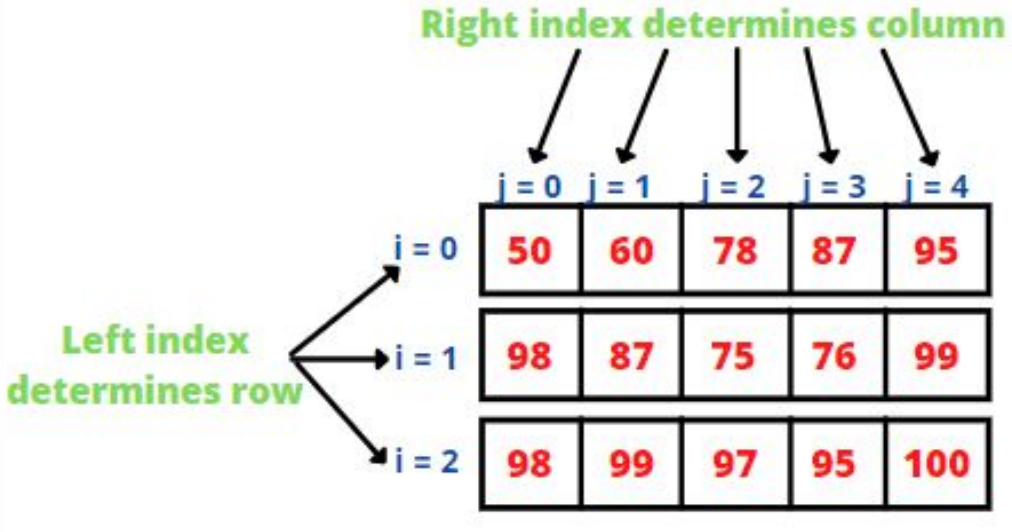

Fig: Arrangement of elements in two dimensional array

	Col1	Col2	Col3	Col4	....
Row1	Arr[0][0]	Arr[0][1]	Arr[0][2]	Arr[0][3]	
Row2	Arr[1][0]	Arr[1][1]	Arr[1][2]	Arr[1][3]	
Row3	Arr[2][0]	Arr[2][1]	Arr[2][2]	Arr[2][3]	
Row4	Arr[3][0]	Arr[3][1]	Arr[3][2]	Arr[3][3]	
⋮					

Contd...

- If you want to store data as a tabular form, like a table with rows and columns, you need to get familiar with multidimensional arrays.
- A multidimensional array is basically an array of arrays.
- Arrays can have any number of dimensions.

Access the Elements of a 2D Array

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
printf("%d", matrix[0][2]);  // Outputs 2
```

Change Elements in a 2D Array

```
int matrix[2][3] = { {1, 4, 2}, {3, 6, 8} };
matrix[0][0] = 9;
printf("%d", matrix[0][0]);  // Now outputs 9 instead of 1
```

Loop through a 2D array

```
int arr[2][3] = { {1, 4, 2}, {3, 6, 8} };
int i, j;
for (i = 0; i < 2; i++)
{
  for (j = 0; j < 3; j++)
  {
    printf("%d\t", arr[i][j]);
  }
      printf("\n");
}
```

Write a program to take input from user and print it using 2D array

```
#include<stdio.h>
int main()
{
int arr[2][3];
int i, j;
printf("Enter the elements of the");
for (i = 0; i < 2; i++)
{
  for (j = 0; j < 3; j++)
  {
    printf(“Enter element %d%d”,i,j);
    scanf("%d", &arr[i][j]);
  }
}
```

```
    for (i = 0; i < 2; i++)
    {
      for (j = 0; j < 3; j++)
      {
            printf("%d\t", arr[i][j]);
      }
      printf("\n");
    }
    return 0;
}
```

Write a program to take input from user and calculate sum and print it using 2D array

```
#include<stdio.h>
int main()
{
    int arr[2][3];
    int i, j, sum = 0;
    printf("Enter the elements of the array");
    for (i = 0; i < 2; i++)
    {
      for (j = 0; j < 3; j++)
      {
        printf("Enter element %d%d",i,j);
        scanf("%d", &arr[i][j]);
      }
    }
    for (i = 0; i < 2; i++)
    {
      for (j = 0; j < 3; j++)
      {
        sum += arr[i][j];
        printf("%d\t", arr[i][j]);
      }
      printf("\n");
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

Program

```
#include<stdio.h>
int main()
{
    int arr[10][10], sum = 0;
    int i, j,m,n;
    printf(“Enter the size of row and column”);
    scanf(“%d%d”, &m, &n);
    printf("Enter the elements of the array");
    for (i = 0; i < m; i++)
    {
      for (j = 0; j < n; j++)
      {
        printf(“Enter element %d%d”,i,j);
        scanf("%d", &arr[i][j]);
      }
    }
    for (i = 0; i < m; i++)
    {
      for (j = 0; j < n; j++)
      {
        sum += arr[i][j];
        printf("%d\t", arr[i][j]);
      }
      printf("\n");
    }
    printf("sum=%d\t", sum);
    return 0;
}
```

Function in C

3. Function in C

- In programming, set of statements that solves a particular task is called function or module.
- Every C program should have at least one function that is main function.

Q) Write a function to check whether a given number is positive or negative.

Example:

```
void check(int num)
{
    if(num == 0)
        printf("Neither positive nor negative\n");
    else if(num > 0)
        printf("Positive\n");
    else
        printf("Negative\n");
}
```

where,

`void` - return type of a function

`check` - function name

`int num` - function parameter. We can give many parameters.

Statements between two curly braces {..} - function body

Why do we need Functions?

1. **Improve Modularity**

   We can divide a large program into multiple small modules.

   If we write programs using modules, it very easy to understand the program.

   And it's also easy to debug (say, which part doesn't work properly) the program.

2. **Code Reusability**

   Let's take an example,

   Get two integer from user and check whether the two numbers are positive or negative.

Contd…

Program without functions

```
#include<stdio.h>
int main()
{
    int num;

    //for first number
    scanf("%d",&num);

    if(num == 0)
        printf("Neither positive nor negative\n");
    else if(num > 0)
        printf("Positive\n");
    else
        printf("Negative\n");

    //for second number
    scanf("%d",&num);

    if(num == 0)
        printf("Neither positive nor negative\n");
    else if(num > 0)
        printf("Positive\n");
    else
        printf("Negative\n");

    return 0;
}
```

- Here, we are writing the same piece of code again and again.
- This is not a good program practice!

Write a function to check whether a given number is positive or negative.

```c
#include<stdio.h>

void check(int num)
{
  if(num == 0)
  {
    printf("Neither positive nor negative\n");
  }
  else if(num > 0)
  {
    printf("Positive\n");
  }
  else
  {
    printf("Negative\n");
  }
}
```

```c
int main()
{
    int num;
    //for first number
    scanf("%d",&num);
    check(num);
    //for second number
    scanf("%d",&num);
    check(num);
    return 0;
}
```

- Here, we wrote a **logic one time** and re-using it wherever it needed again.
- This is how function improves **code reusability** in programming.

Write a functions to check whether a given number is even or odd.

```c
#include<stdio.h>

void even_odd(int num)
{
    if(num%2 == 0)
    {
        printf("The number is even\n");
    }
    else
    {
        printf("The number is odd\n");
    }
}
```

```c
int main()
{
    int num;
    printf("\nEnter a number");
    scanf("%d",&num);
    even_odd(num);
    printf("\nEnter a number");
    scanf("%d",&num);
    even_odd(num);
    printf("\nEnter a number");
    scanf("%d",&num);
    even_odd(num);

    return 0;
}
```

Write 2 functions to check whether a given number is positive or negative and also even or odd.

```c
#include<stdio.h>
void check(int num)
{
  if(num == 0)
  {
    printf("Neither positive nor negative\n");
  }
  else if(num > 0)
  {
    printf("Positive\n");
  }
  else
  {
    printf("Negative\n");
  }
}
```

```c
void even_odd(int num)
{
    if(num%2 == 0)
    {
        printf("The number is even\n");
    }
    else
    {
        printf("The number is odd\n");
    }
}

int main()
{
    int num;
    scanf("%d",&num);
    check(num);
    even_odd(num);
    return 0;
}
```